

学习资源

记忆机制

 Copy page

记忆机制允许 Coding Agent 在多个任务和会话之间保存上下文信息，从而减少重复输入并提升任务执行效率。通过合理的记忆设计，Agent 可以持续理解项目结构、开发规范和用户偏好，并在后续任务中自动复用这些信息。

在 Coding Agent 系统中，记忆通常分为**自动记忆**、**项目记忆**和**会话记忆**等不同层级。

为什么 Coding Agent 需要记忆机制？

传统的大模型在每次调用时不会保留之前的状态，因此存在无法跨会话记住项目背景、无法积累解决问题的经验、无法持续学习用户偏好等问题。

Agent 系统通过 **外部记忆（External Memory）** 解决这一问题。

典型架构如下：

```
graph TD; A[User input] --> B[Memory retrieval]; B --> C[Context assembly]; C --> D[LLM reasoning]; D --> E[Action / tool call]; E --> F[Memory update];
```

也就是说：Agent 在执行任务前会检索记忆，在任务结束后更新记忆。



这种架构是现代 Agent 系统（LangGraph、AutoGPT、Devin 等）的常见设计。

>

现代 Coding Agent 的完整记忆架构

在架构上，一个完整的 Agent Memory Architecture 通常如下：

Short-term memory



Session context

Long-term memory

├ semantic memory

├ episodic memory

└ procedural memory

Coding Agent 的主要记忆类型

Session Memory

Project Memory

Semantic Memory

Episodic Memory

Procedural Memory

Session Memory 是当前任务中的上下文信息。包括当前对话历史、最近工具调用结果、当前执行计划、当前文件内容。这些信息通常存在于模型上下文窗口（context window）中。特点是生命周期仅为单个会话、容量受上下文限制、不会长期保存。

例如：

User：修复这个 Python bug

Agent：分析错误

Agent：修改代码

Agent：运行测试

这些执行步骤都属于会话记忆。

在实际工程中，Agent 使用记忆通常遵循固定流程。

>

1 记忆检索

在任务开始前，Agent 会检索项目记忆，知识库，历史经验并注入到上下文中。

2 上下文构建

检索到的记忆会被拼接成完整上下文，然后输入到模型。

3 记忆更新

任务完成后，Agent 会决定是否写入新记忆，例如：新的项目规则、新的调试经验、用户偏好.....

如何正确使用 Coding Agent 的记忆机制

主流 Agent 系统中的记忆机制通常具有 **分层、可控、可检索、可更新** 等特点。

✓ 一般而言，系统会将记忆划分为 **短期记忆** 与 **长期记忆** 两类：短期记忆主要用于保存当前线程或会话中的状态信息，而长期记忆则需要通过显式文件、规则配置、向量检索或其他持久化存储机制进行维护。

① 以 **Claude Code** 为例，其官方文档明确说明：每个会话都会从一个新的上下文窗口开始，跨会话的信息延续主要依赖 **CLAUDE.md** 等持久化指令文件以及自动记忆（auto memory）机制。类似地，在 **LangChain / LangGraph** 中，memory 也被划分为 **thread-scoped** 的短期记忆与 **跨会话的长期记忆**。

在实际使用 Agent 系统时，更有效的方式并不是依赖模型“自动记住一切”，而是建立一套清晰的记忆管理范式。例如：哪些信息应写入项目级记忆文件，哪些信息适合通过知识库或向量检索获取，哪些信息只需保留在当前会话中，以及在任务结束后哪些内容值得沉淀为长期记忆。

通用 Coding Agent 最容易落地的第一原则，是先区分两种完全不同的记忆：

- **指令型记忆**：它由人来写，目的是告诉 Agent “应该如何做事”。这类内容通常包括编码规范、目录约定、构建命令、测试流程、命名风格、提交要求、团队安全规则等。Claude Code 里对应的是 `CLAUDE.md` 这一类持久指令文件。
- **学习型记忆**：它不是你预先规定的，而是 Agent 在执行过程中，从你的纠正、偏好、失败经验、常见命令和项目习惯中逐步积累出来的。Claude Code 文档把这类能力称为“自动记忆”，并说明它会在每次会话开始时与指令型记忆一起加载。对于 subagent，它还可以维护自己的独立记忆目录，并在系统提示中自动包含 `MEMORY.md` 的前 200 行。

💡 如果将这两类记忆混为一体，往往会导致系统行为逐渐偏离预期。更合理的做法是对记忆进行明确分工：

- 将 **规范、制度与行为约束** 写入 **指令型记忆 (instruction memory)**，用于稳定约束 Agent 的行为；
- 将 **经验、用户偏好、临时发现以及复盘结论** 写入 **学习型记忆 (learning memory)**，用于在后续任务中持续改进决策。

这种区分可以避免经验性信息不断污染系统规则，从而保持 Agent 行为的稳定性与可控性。

* 分层管理

- 组织级记忆
- 项目级记忆
- 用户级记忆
- 本地级记忆
- 子代理 / 角色级记忆

* 按路径加载 `.md` 文件

Claude Code 官方文档中针对大型项目的记忆组织方式非常具有参考价值。对于规模较大的代码库，建议将规则拆分到 `.claude/rules/` 目录下的多个 Markdown 文件中，每个文件只关注一个主题，例如 `testing.md`、`api-design.md`、`security.md` 等。

更重要的是，这些规则可以根据 **文件路径或子目录进行范围限定**。只有当 Agent 正在处理匹配路径的文件时，对应规则才会被加载到上下文中。这样既能减少无关规则带来的噪音，也可以有效节省上下文空间。

如果将这一做法抽象为一种通用的 Coding Agent 设计模式，可以总结为三条原则：

- **主记忆文件只保留全局共识**，例如项目背景、整体架构或全局约定；
- **专项规则尽量模块化**，按主题拆分为独立规则文件；
- **能够按路径加载的规则，不要全量加载**，尽量在需要时再引入上下文。

基于这一思路，可以将项目的记忆结构设计为如下形式：

```
agent-memory/  
├─ project.md          # 项目总说明  
├─ rules/  
│   ├─ code-style.md   # 代码风格  
│   ├─ testing.md      # 测试约定  
│   ├─ api-design.md   # API 设计规范  
│   ├─ security.md     # 安全要求  
│   └─ frontend/  
│       └─ react.md     # 前端专项规范  
└─ local/  
    └─ developer.local.md
```

✓ 这种结构有三个好处：

1. 便于维护。每份规则只负责一个主题，不容易越写越乱。Claude Code 文档明确建议每个规则文件覆盖一个主题，并使用有描述性的文件名。
2. 便于按需加载。Agent 在处理测试文件时，不必把前端规范和数据库规则都塞进上下文。
3. 便于团队协作。不同小组可以各自维护自己的规则目录，而不是争抢同一个主文件。

编写 Agent 记忆时，应尽量使用 **具体、可验证的规则**，而不是抽象的原则。指令越明确，Agent 的行为越稳定。

一般建议：

- 指令表达 **简洁、明确**；
- 规则之间 **保持一致性**；
- 主记忆文件最好控制在 **200 行以内**；
- 使用 **Markdown 标题与列表结构** 提升可读性；
- 尽量将要求写成 **可检查、可执行的规则**。

⚠️ 例如，不推荐写成：

- 保持代码整洁
- 做好测试
- 注意 API 设计
- 适当拆分模块

✓ 而更推荐写成：

- 所有新增 **TypeScript** 文件使用 **2 空格** 缩进
- 修改业务逻辑后 **必须运行** `pnpm test`
- API handler **统一放在** `src/api/handlers/`
- **React** 页面组件 **不超过 300 行**，超过则拆分为 hooks 或子组件

具体规则能够显著降低 Agent 的解释空间，从而提高执行一致性。

* 将共享规则与个人偏好分开

在设计 Agent 记忆结构时，需要明确 **不同规则的作用范围与责任主体**。一个常见做法是按作用域进行分层：

- **项目级 (Project)** ：所有团队成员共享，并通过版本控制维护



• **组织级 (Organization)** : 由 IT 或 DevOps 统一制定, 例如安全规范或开发流程

• **用户级 (User)** : 仅对个人生效, 例如个人编码习惯



• **本地级 (Local)** : 只适用于当前机器或当前工作环境, 不进入 Git

• **角色级 (Role / Agent-specific)** : 仅供某类专用 Agent 使用

这一分层的核心原则是:

谁负责、谁共享、谁生效。

例如:

- 团队统一规范 → 项目级
- 公司安全策略 → 组织级
- 个人代码习惯 → 用户级
- 当前机器配置 → 本地级
- 某个专用 Agent 的规则 → 角色级

在记忆设计阶段就明确这些边界, 可以避免规则混乱或重复定义。

* 通过导入机制与规则包实现复用

在实际项目中, 大量规则是 **跨仓库共享的工程规范**。如果每个仓库都重复编写, 不仅维护成本高, 也容易出现不一致。

以 **Claude Code** 为例, 其文档说明:

- 可以在 `CLAUDE.md` 中使用 `@path/to/import` 导入其他规则文件;
- `.claude/rules/` 支持通过 **符号链接 (symlink)** 共享规则;
- 导入内容可以 **递归展开**, 符号链接会被正常解析。

这使得团队可以构建 **可复用的规则包 (rule packages)** , 例如:

- `company-security-rules`
- `frontend-react-rules`

每个项目只需引用需要的规则模块，而无需重复维护整套规范。

这种做法带来两个直接收益：

1. 规则可以集中维护与统一更新；
2. 不同项目共享相同的工程语言，使 Agent 在多个仓库中的行为更加一致。

记忆故障排除

▼ Coding Agent 没有遵循我的`.md`记忆文件

`.md` 记忆文件通常作为上下文提示 提供给 Agent，而不是强制配置。

Agent 会读取这些内容并尽量遵循，但对于 模糊、不明确或互相冲突的规则，并不能保证严格执行。

如果 Agent 没有遵循规则，可以按以下方式排查：

- 运行 `/memory`（或等效命令）确认 `.md` 记忆文件是否被加载。
- 检查 `.md` 文件是否位于 当前会话允许加载的路径或作用域。
- 检查是否存在 多个 `.md` 文件之间的冲突规则。如果不同文件对同一行为给出不同指令，Agent 可能会随机选择其中之一。

▼ 我不知道自动记忆保存了什么

大多数 Coding Agent 会在后台维护自动记忆，用于记录项目背景、用户偏好或常见操作。

可以通过以下方式查看：

- 运行 `/memory`（或类似命令）查看当前自动记忆目录。
- 自动记忆通常以 Markdown 文件形式存储，可以直接阅读、修改或删除。

过大的记忆文件会占用更多上下文窗口、降低 Agent 对规则的遵循度，增加冲突概率。

建议将详细说明拆分为多个 Markdown 文件，使用文件引用或导入机制（例如 `@path/to/file` ）或将规则拆分到专门的规则目录中（例如 `rules/` ）

▼ 指令在上下文压缩后消失

许多 Coding Agent 在长对话中会执行上下文压缩或摘要，以减少上下文长度。

通常情况下记忆文件会在压缩后重新从磁盘加载，只有写入记忆文件的内容才会持续存在。如果某些规则在压缩后消失，说明这些规则只存在于对话中并没有被写入记忆文件

解决方法：

- 将长期需要的指令写入 `.md` 记忆文件
- 不要只在对话中说明规则

学习资源



Coding Agent 工作原理



Agentic 扩展组件



常用工作流



最佳实践

